

EXERCISE 8

RSA CRYPTOSYSTEM

1. One major drawback of public-key algorithms is that they are relatively slow. In Section 7.5.1 we learned that an acceleration technique is to use short exponents e . We study short exponents in this problem in more detail.

- (1) Assume that in an implementation of the RSA cryptosystem one modular squaring takes 75% of the time of a modular multiplication. How much quicker is one encryption on average if instead of a 2048-bit public key the short exponent $e = 2^{16} + 1$ is used? Assume that the square-and-multiply algorithm is being used in both cases.
- (2) Most short exponents are of the form $e = 2^n + 1$. Would it be advantageous to use exponents of the form $2^n + 1$? Justify your answer.
- (3) Compute the exponentiation $x^e \bmod 29$ with $x = 5$ and both variants of e from Part 2 of this problem with $n = 4$. Use the square-and-multiply algorithm and show each step of your computation.

2. One of the most attractive applications of public-key algorithms is the establishment of a secure session key for a private-key algorithm such as AES over an insecure channel.

Assume Bob has a pair of public/private keys for the RSA cryptosystem. Develop a simple protocol using RSA which allows the two parties Alice and Bob to agree on a shared secret key. Who determines the key in this protocol, Alice, Bob, or both?

3. In practice, it is sometimes desirable that both communication parties influence the selection of the session key. For instance, this prevents the other party from choosing a key which is a weak key for a symmetric algorithm. Some block ciphers such as DES and IDEA have weak keys. Messages encrypted with weak keys can be recovered relatively easily from the ciphertext.

Develop a protocol similar to the one above in which both parties influence the key. Assume that both Alice and Bob have a pair of public/private keys for the RSA cryptosystem. Please note that there are several valid approaches to this problem. Show just one.

4. In this exercise, you are asked to attack an RSA-encrypted message. You are the attacker and you obtain the ciphertext $y = 1141$ by eavesdropping on the channel. The public key is $k_{pub} = (n, e) = (2623, 2111)$.

- (1) Consider the encryption formula. All variables except the plaintext x are known. Why can't you simply solve the equation for x ?
- (2) In order to determine the private key d , you have to calculate $d \equiv e^{-1} \bmod \Phi(n)$. There is an efficient expression for calculating $\Phi(n)$. Can we use this formula here?
- (3) Calculate the plaintext x by computing the private key d through factoring n . (Hint: Factorization for such small RSA moduli can be done through an exhaustive search with a list of all small primes and a simple program that checks which prime up

to $\sqrt{n}$ factors n .) Does this approach remain suitable for numbers with a length of 2048 bits or more?

5. We now show how an attack with chosen ciphertext can be used to break an RSA encryption.

- (1) Show that the multiplicative property holds for RSA, i.e., show that the product of two ciphertexts y_1 and y_2 is equal to the encryption of the product of the two respective plaintexts x_1 and x_2 . Under certain circumstances, this property might be exploited by an attacker.
- (2) Assume Oscar eavesdrops and obtains a ciphertext y_1 , which is the encrypted version of a message x_1 that was sent from Alice to Bob. Oscar would like to know the plaintext x_1 . Oscar computes an innocent looking message t , which he encrypts. We assume that he can obtain the decryption of one ciphertext that he sends to Bob, e.g., by having access to Bob's computer at a certain point in time. Show how Oscar can construct a ciphertext y in such a way that he can use its decryption for computing x_1 .

6. In this exercise, we illustrate the problem of using nonprobabilistic cryptosystems, such as schoolbook RSA, imprudently. Nonprobabilistic means that the same plaintext maps to the same ciphertext. This allows traffic analysis (i.e., to draw some conclusion about the plaintext by merely observing the ciphertext) and in some cases even the total break of the cryptosystem. The latter holds especially if the number of possible plaintexts is small. Suppose the following situation:

Alice wants to send a message to Bob encrypted with his public key (n, e) . She decides to use the ASCII table to assign a number to each character (space $\rightarrow 32$, ! $\rightarrow 33$, ..., A $\rightarrow 65$, B $\rightarrow 66$, ..., ~ $\rightarrow 126$) and to encrypt them separately.

- (1) Oscar eavesdrops on the transferred ciphertext. Describe how he can successfully decrypt the message by exploiting the nonprobabilistic property of RSA.
- (2) Bob's RSA public key is $(n, e) = (3763, 11)$. Decrypt the ciphertext

$$y = 2514, 1125, 333, 3696, 2514, 2929, 3368, 2514$$

with the attack proposed in 1. For simplification, assume that Alice only chose capital letters A, ..., Z during the encryption.

- (3) Is the attack still possible if we use OAEP padding? Exactly explain your answer.

7. There is a vulnerability in RSA if the primes are re-used. Assume a certificate server that generates RSA key pairs $k_{\text{pub}} = (n, e)$, $k_{\text{pr}} = d$ for users. Of course, to generate the public keys, the server computes $n = pq$, where p and q are two large primes. The server does not disclose the primes to the outside world, not even to the owner of the private key.

Two users, User 1 and User 2, obtain the key pairs $k_{\text{pub1}} = (n_1, e_1)$, $k_{\text{pr1}} = d_1$ and $k_{\text{pub2}} = (n_2, e_2)$, $k_{\text{pr2}} = d_2$. Due to a faulty true random generator, n_1 was computed as $n_1 = p_1 q_1$ and n_2 as $n_1 = p_1 q_2$, i.e., p_1 was used for both key pairs.

- (1) Describe how an attacker can compute both private keys if he has both public keys, which are, of course, publicly known. (Hint: You have to use the Euclidean algorithm.)

- (2) You look up the two public keys $k_{\text{pub1}} = (4757, 17)$ and $k_{\text{pub2}} = (2059, 129)$ in a certificate database. By wiretapping, you observe that User 1 receives the ciphertext $y_1 = (0, 1884, 429)$ and User 2 receives $y_2 = (1186, 836, 0, 1114, 137, 46)$. Your task is to break the system and to recover the actual letters that form the two plaintexts x_1 and x_2 . The mapping between letters and plaintext numbers is simple (cf. also Table 1.3):

$$A \rightarrow 0, B \rightarrow 1, \dots, Z \rightarrow 25$$

Each letter is individually encrypted with RSA, i.e., the first plaintext consists of three letters and the second one of six. (Remark: In this basic form, RSA encrypts the plaintext value 0 to the ciphertext 0, which is a weakness. In practice, one uses additional techniques, especially padding, to prevent this.)

8. By coincidence you learn that your colleagues are planning a secret party at somebody's house and you are not invited. Not very nice! In order to go to the correct place you eavesdrop on your colleagues. Luckily, you manage to obtain the email that your dear colleagues exchange. Unfortunately, this email is RSA encrypted. You have the following information: $k_{\text{pub}} = (n, e) = (1271, 11)$ Ciphertext: [955, 458, 562, 911, 914, 269, 690]

The plaintext is the ASCII encoded name of the host of the party. Each ciphertext contains one encrypted ASCII symbol. Break RSA through factoring, find the location of the party, and surprise your colleagues!